

《微处理器与接口技术课程设计》

报告

——多量程电压表设计

专 业： 电子信息工程

班 级： 1412

姓 名： 孙玮

学 号： 14200106214

2016 年 7 月

1. 课程设计目的

- 1.1 巩固和加深对微处理器原理和接口技术知识的理解；
- 1.2 培养根据课题需要选学参考书籍、查阅手册和文献资料的能力；
- 1.3 学会方案论证的比较方法，拓宽知识，初步掌握工程设计的基本方法；
- 1.4 掌握常用仪器、仪表的正确使用方法，学会软、硬件的设计和调试方法；
- 1.5 能按课程设计的要求编写课程设计报告，能正确反映设计和实验成果，能用计算机绘制电路图和流程图。

2. 课程设计要求

- 2.1 使用 ADC0832 进行模数转换；
- 2.2 通过串口读取测量到的电压值；
- 2.3 数码管显示测量到的电压值；
- 2.4 可测多个量程的电压。

3. 硬件设计

3.1 设计思想

该系统的硬件设计思路如下：首先，通过 ADC0832 对外部电压进行模数转换，然后通过时钟线、数据线和控制使能线与单片机相连。而后，单片机通过八根数据线与四个控制线与四位数码管进行连接，用于显示测量到的电压。另外，单片机通过 TX 给外部发送数据，PC 端可以通过 USB-TTL 模块直接接收到电压值。最后，关于多量程设计，量程方面我采用了电阻分压方式，这样就可以实现大量程的测量，量程选择方面我使用了开关与继电器实现。

3.2 主要元器件介绍

3.2.1 ADC0832

ADC0832 是美国国家半导体公司生产的一种 8 位分辨率、双通道 A/D 转换芯片。芯片引脚图如图 1 所示。其引脚分别为 CS（片选使能）、CH0（模拟输入通道 0）、CH1（模拟输入通道 1）、GND（供电及参考电压低电位）、DI（数据信号输入）、DO（数据信号输出）、CLK（时钟信号输入）以及 VCC/VREF（电源及参考电压输入）其时序逻辑图如图 2 所示。由图 2 可知一般情况下 ADC0832 与单片机连接应使用四根控制信号线，分别为 CS、DO、DI 和 CLK。但由图 2 可知 DO 与 DI 其中一个作用时，另一个则不作用，因此在进行硬件电路设计时可以将 DO 和 DI 连接在同一个 I/O 口上。

ADC0832 为 8 位分辨率 A/D 转换芯片，其最高分辨可达 256 级，可以适应一般的模拟

量（低精度）转换要求。

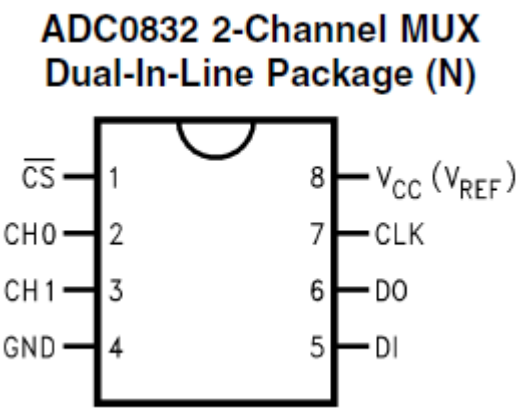


图 1

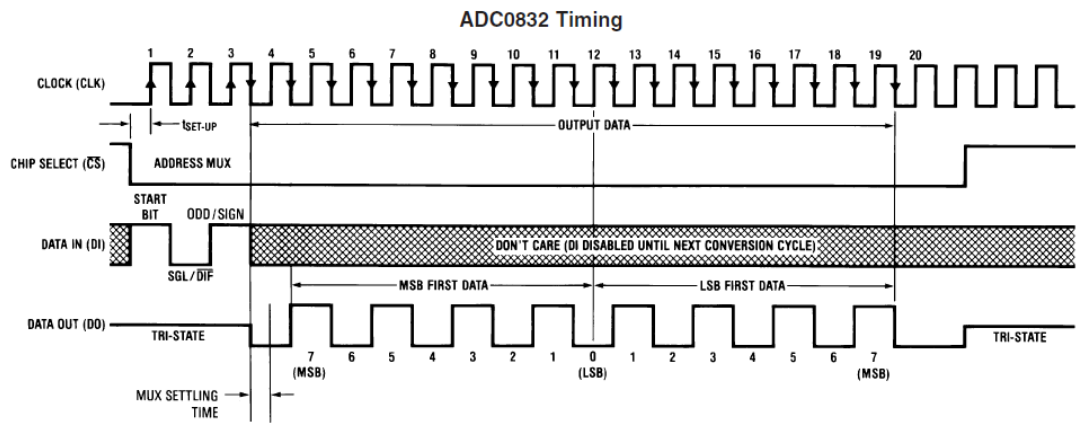


图 2

3. 3. 2 STC89C52RC

STC89C52RC 单片机是宏晶科技推出的超强抗干扰、高速、低功耗单片机,指令代码完全兼容传统 8051 单片机。其最高频率时钟为 80 MHz, Flash 存储器为 8KB, RAM 为 512 bit,E2PROM 为 2KB,可反复擦写编程。工作温度为-40℃~+85℃, 内置看门狗电路,内部电源供电系统、时钟电路和复位电路都经过特殊处理。“6 时钟/机器周期”和“12 时钟/机器周期”可在 ISP 编程时反复设置。其 DIP 封装引脚图如图 3 所示。实物图如图 4 所示。

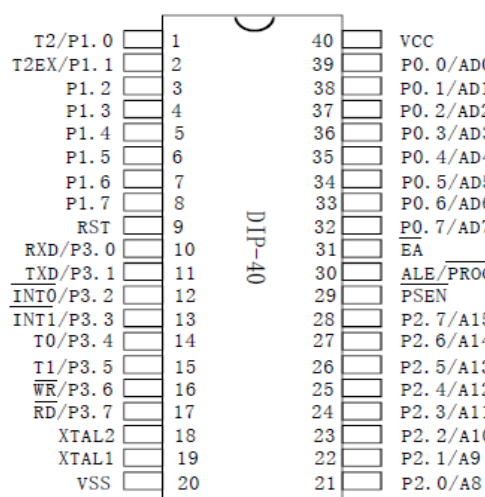


图 3



图 4

3.2.3 四位共阳极数码管

图 5 所示即为该系统所采用的共阳极数码管。该数码管共有 12 个引脚，其中 8 个为数据引脚，4 个为控制引脚。数据引脚作用为向数码管输入要显示的数据，控制引脚作用为分别控制四个数码的亮和灭。图 6 所示及其内部电路图。

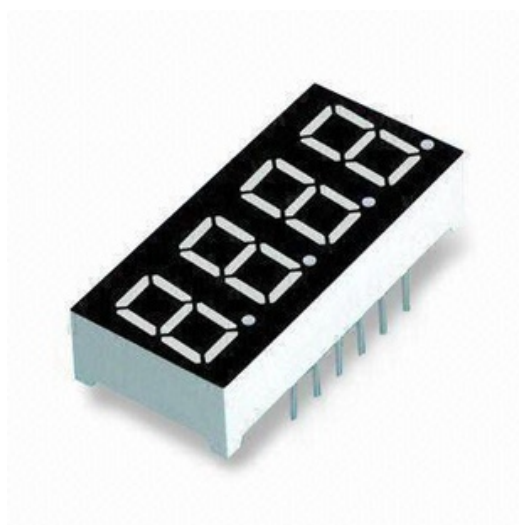


图 5

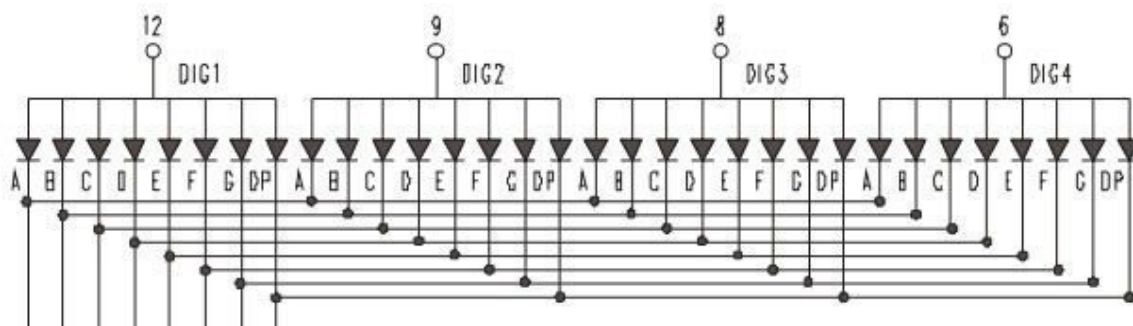


图 6

3.3 原理图及功能电路介绍

整体原理图如图 7 所示。该电路可以分为三个部分，分别为：模数转换部分、显示部分和数据处理部分。

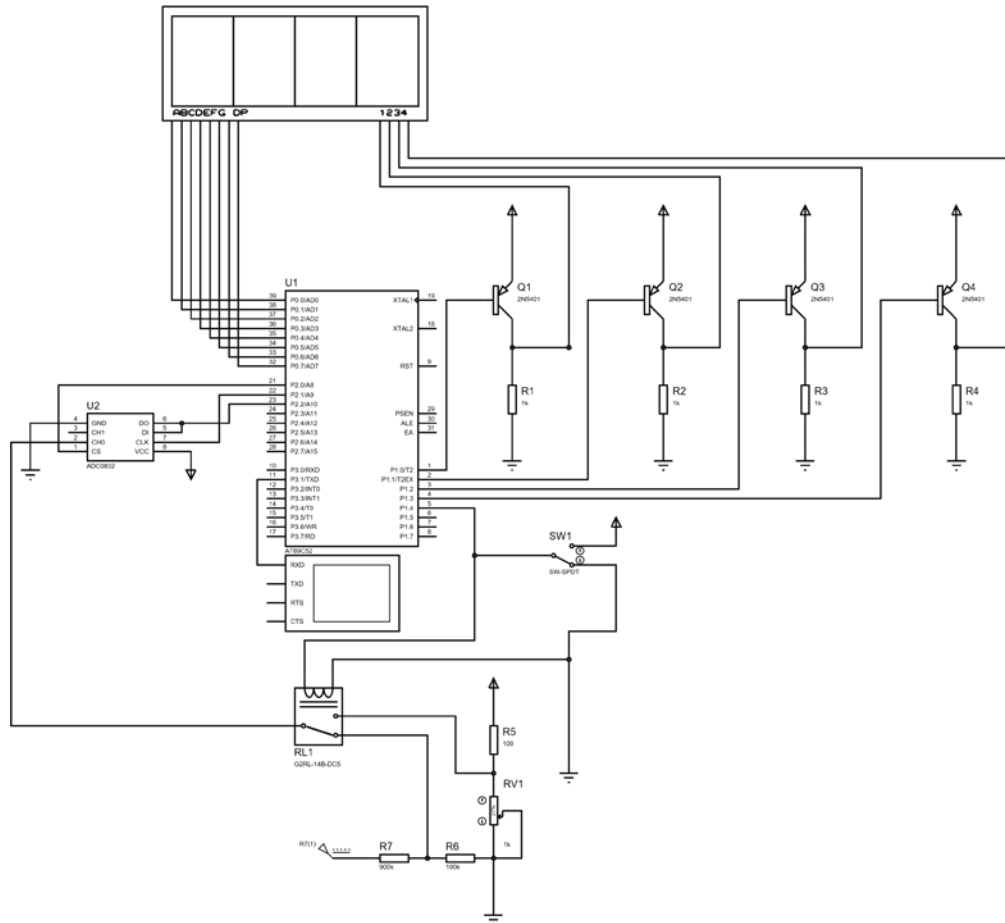


图 7

3.3.1 显示部分

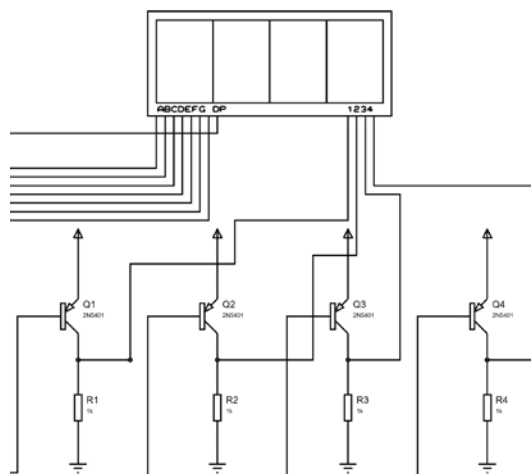


图 8

图 8 所示即为显示部分电路图。该部分实现的功能是显示测量到的树值。数码管的八个的数据引脚引出接到单片机的 P0 上，用于接受显示数据。四个控制引脚分别接在四个 PNP 管的集电极上，然后 PNP 管的基极接单片机的四个 I/O 口，发射极接 VCC。这里控制引脚不直接接单片机上的原因是：单片机 I/O 无法提供大电流，不能作为共阳极数码管的控制信号，因此需要接三极管方大电流。

3.2.2 模数转换部分

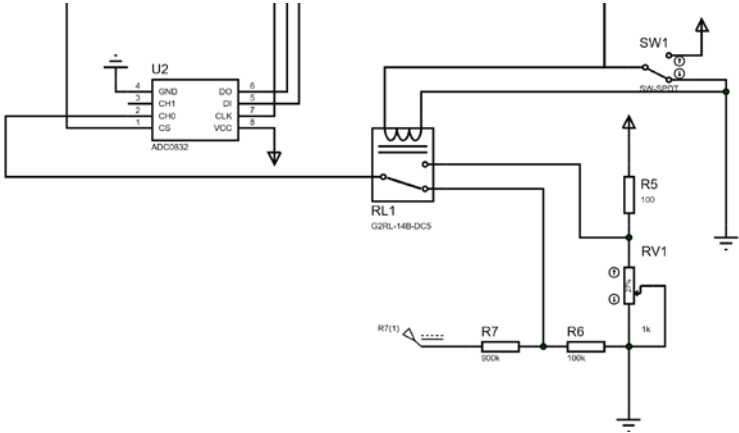


图 9

图 9 所示即为模数转换部分电路图。该部分的主要作用是测量电压值并以数字量传输到单片机中。该部分可以分为两个子部分，分别为测量部分和量程控制部分。测量部分由 ADC0832 组成，它通过三根线将 CS、CLK、DO 与 DI 与单片机相连。另外，其 CH0 接到量程控制部分读取电压。量程控制部分主要由继电器、开关和电阻组成。开关连接一个 I/O 口和继电器。继电器选择接入 CH0 的信号，当为导通时，量程为 0~5V，外部无其他电路，当为不导通时，量程为 0~50V，外部接一个电阻分压电路。

3.2.3 数据处理部分

中央数据处理部分由 89C52RC 构成。该部分将上述两部分整合并控制。MCU 的 P0 口接数码管的数据线；P1.0~P1.3 接数码管控制部分（三极管的基极）；P1.4 接量程控制部分的开关，作用是读取当前量程选择情况；P2.0~P2.2 分别接 ADC0832 的 CS、CLK 和 DO/DI；TX 接外部通信设备，接受当前的电压测量值。

4. 软件设计

4.1 设计思想

单片机启动后，先初始化串口，并定义缓存变量，然后进行 while 死循环。在死循环中，先调用 AD 转换函数读一次“电压值”，但这时得到的并不是真正的电压值，所以需要进行精

度修正，将范围 0~255 的范围修正为 0~500。再判断当前量程选择情况，根据它来判断显示与串口数据的小数点位置，最后将数据显示并通过串口发送。

4.2 软件流程图

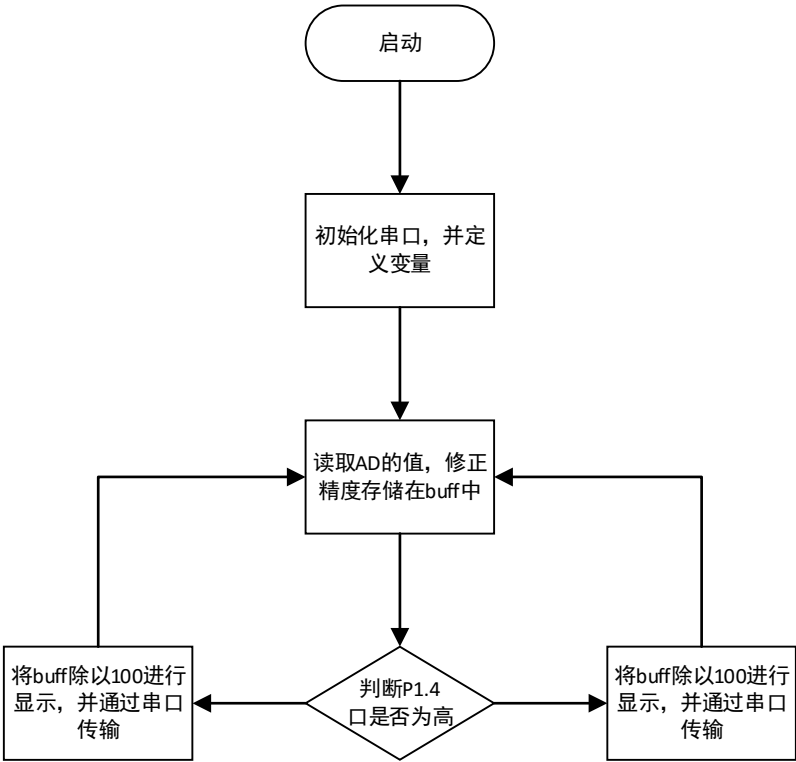


图 10

4.3 源程序

```
#include <reg52.h> //52 系列单片机头文件定义
#include <stdio.h> //C 语言头文件定义

//*****
//*****引脚定义*****
//*****

//ADC0832 控制口定义
sbit CS =P2^0; //CS: 片选使能信号输出端
sbit CLK=P2^1; //CLK: 时钟信号输出端
sbit DIO=P2^2; //DI/D0 信号输出端（由于 D0 与 D1 在通信时不会同时有效，
所以共用一个 I/O 口）

//数码管控制引脚定义（数据引脚共 8 个，使用 P0）
sbit S_1=P1^0; //第一个数码管阳极控制端
sbit S_2=P1^1; //第二个数码管阳极控制端
sbit S_3=P1^2; //第三个数码管阳极控制端
sbit S_4=P1^3; //第四个数码管阳极控制端
```

```

//量程与继电器控制引脚定义
sbit KEY=P1^4; //用于量程的选择以及继电器的控制

//*****
//*****常量定义*****
//*****
unsigned int table[]={0xc0,0xf9,0xa4,0xb0,0x99,0x92,0x82,0xf8,0x80,0x90};
//0~9 在共阳数码管中所对应的十六进制值

//*****
//*****延时 1000us*****
//*****
void delay_1000us() //简单延时函数,约延时 1000us
{
    unsigned char i=1000;
    while(i--);
}

//*****
//*****ADC0832 读取程序*****
//*****
unsigned int ADC0832() //此处编程参考 ADC0832 英文手册
{
    unsigned int i, buff; //缓存数据定义

    //一个读取周期开始
    CS=1; //CS 置一, 转换开始
    CLK=0; //为第一个时钟信号做准备
    CS=0; //芯片使能
    DIO=1; //给发送 DI 发送起始电平
    CLK=1; //第一个时钟开始
    CLK=0; //第一个时钟结束
    DIO=1; //给 DI 发送高电平, 选择单通道模式
    CLK=1; //第二个时钟开始
    CLK=0; //第二个时钟结束
    DIO=0; //给 DI 发送低电平, 选择通道 0
    CLK=1; //第三个时钟开始
    CLK=0; //第三个时钟结束
    DIO=1; //DO 置一, 准备读取数据
    for(i=0; i<8; i++) //读取八位数据, 由高到低读取
    {
        CLK=1; //时钟信号拉高
        CLK=0; //下降沿有效, 进行读取数据
        buff<<=1; //将下面储存的低位数据向右移
        buff|=DIO; //将输出数据 DIO 通过或运算储存在 buff 最低位
    }
}

```

```

        CS=1;                                //测量完毕，将 CS 拉高，片选无效
        return buff;
    }

//*****
//*****串口初始化程序*****
//*****

void UartInit(void)                          //串行口初始化
{
    SCON=0x50;                               //8 位数据,可变波特率
    TMOD=0x20;                               //设定定时器 1 为 16 位自动重装方式
    TL1=0xFD;                               //设定定时初值
    TH1=0xFD;                               //设定定时初值
    ET1=0;                                   //禁止定时器 1 中断
    TR1=1;                                   //启动定时器 1
}

//*****
//*****主程序*****
//*****

void main()
{
    unsigned int buff, ge, shi, bai;         //缓存数据定义
    UartInit();                             //串口初始化
    while(1)                                //死循环
    {
        buff=ADC0832()*100;                 //读取 AD 的值，并进行修正
        buff=buff/51;                       //精度修正，转换为 000~500 范围
        bai=buff/100;                       //保存 buff 的百位
        shi=(buff%100)/10;                  //保存 buff 的十位
        ge=buff%10;                         //保存 buff 的个位
        //数码管显示程序
        if(KEY==1)                          //如果 KEY 为 1，量程设为 0~5V
        {
            S_1=0;                          //选通第一位数码管，显示第一位数
            P0=table[0];                     //P0 口输出 8 位数据
            delay_1000us();                  //短暂延时
            P0=0xff;                         //数码管消隐
            S_1=1;                          //关闭第一位数码管

            S_2=0;                          //选通第二位数码管，显示第二位数
            P0=table[bai]&0x7f;              //P0 口输出 8 位数据，因为量程为 0~5V，所以第一位后显
示小数点
            delay_1000us();                  //短暂延时

```

```

P0=0xff;          //数码管消隐
S_2=1;           //关闭第二位数码管

S_3=0;           //选通第三位数码管，显示第三位
P0=table[shi];   //P0 口输出 8 位数据
delay_1000us();  //短暂延时
P0=0xff;         //数码管消隐
S_3=1;           //关闭第三位数码管

S_4=0;           //选通第四位数码管，显示第四位
P0=table[ge];    //P0 口输出 8 位数据
delay_1000us();  //短暂延时
P0=0xff;         //数码管消隐
S_4=1;           //关闭第四位数码管
//串口发送程序
TI=1;            //TI 置一，开始发送数据
printf("%d.%d%d\n", bai, shi, ge);
//将 buff/100 后发出
while(!TI);      //直到数据发送完毕
TI=0;            //数据发送完毕后，TI 置0
}
else             //如果 KEY 为 1，量程设为 0~50V
{
S_1=0;           //选通第一位数码管，显示第一位
P0=table[0];     //P0 口输出 8 位数据
delay_1000us();  //短暂延时
P0=0xff;         //数码管消隐
S_1=1;           //关闭第一位数码管

S_2=0;           //选通第二位数码管，显示第二位
P0=table[bai];   //P0 口输出 8 位数据
delay_1000us();  //短暂延时
P0=0xff;         //数码管消隐
S_2=1;           //关闭第二位数码管

S_3=0;           //选通第三位数码管，显示第三位
P0=table[shi]&0x7f; //P0 口输出 8 位数据，因为量程为 0~50V，所以第二位后
显示小数点
delay_1000us();  //短暂延时
P0=0xff;         //数码管消隐
S_3=1;           //关闭第三位数码管

S_4=0;           //选通第四位数码管，显示第四位
P0=table[ge];    //P0 口输出 8 位数据

```

```

delay_1000us();          //短暂延时
P0=0xff;                 //数码管消隐
S_4=1;                   //关闭第四位数码管
                           //串口发送程序
TI=1;                    //TI 置一，开始发送数据
printf("%d%d.%d\n", bai, shi, ge);
                           //将 buff/100 后发出
while(!TI);              //直到数据发送完毕
TI=0;                    //数据发送完毕后，TI 置0
}
}
}

```

5. 调试运行

5.1 小量程测量结果

图 11 为小量程电压的仿真测量结果。左下方电压表为 ADC0832 输入的电压，为 4.32V。而图中数码管读到的值为 4.33V，串口发送的数据也为 4.33V。因此，可以得出结论，在小量程的仿真测试中，电压测量误差非常小，可忽略不计。

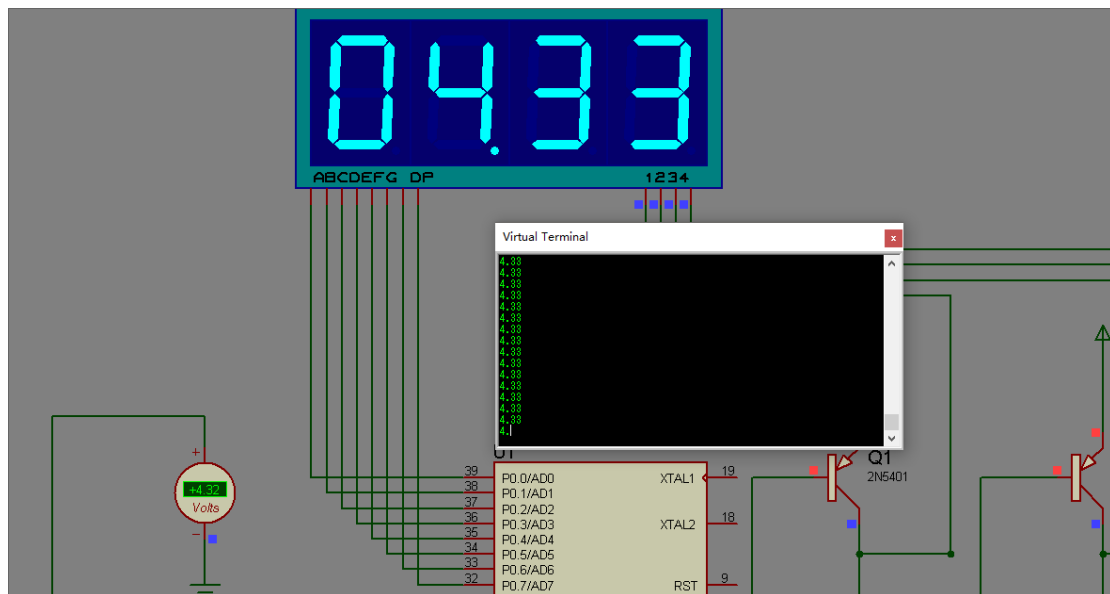


图 11

5.1 大量程测量结果

图 12 为大量程电压的仿真测量结果。左下方第一个电压表为 ADC0832 输入的电压，为 4.52V，第二个电压表为所测激励源的电压，为 45.2。而图中数码管读到的值为 45.2V，串口发送的数据也为 45.2V。因此，可以得出结论，在大量程的仿真测试中，电压测量误差几乎没有。

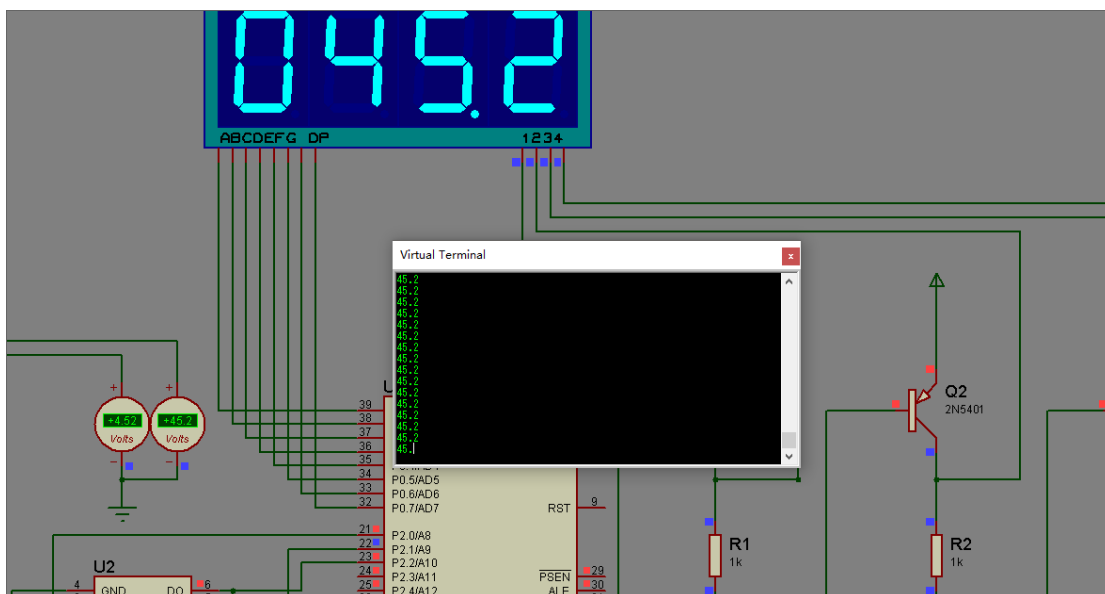


图 12

6. 设计心得体会

通过此次课程设计, 让我们对所学的 51 单片机更加熟悉了解, 对理论学习时没有掌握牢固的一些知识以及一些在学习中存在的漏洞进行学习并加以弥补。在学习上光有理论知识那只是“纸上谈兵”, 还需实际动手去实践。真正把所学的用到日常生活中, 理论联系实际, 做出实物模型。这次单片机课程设计, 我设计的是多量程电压表设计, 通过这次课程设计我感觉到要想做成功, 必须花时间多做准备, 查阅大量资料, 每个过程都很繁琐, 都要认真地分析每一步每一个模块要实现大的功能, 然后分步进行编写调试, 最后整合成在一起。另外, 通过本次课程设计, 我也学会了熟练使用 proteus 进行仿真设计。

在这次课程设计, 我们知道了基于单片机的系统设计, 需要从两个方面入手: 一是软件的实现即程序的编写, 二是硬件电路的实现。因此在这次的单片机课程设计中, 我不仅巩固了我所学的单片机知识, 而且也提高了我的硬件设计水平和动手能力, 使得我的实践能力进一步增强。

参考书目:

- [1] 肖金球. 单片机原理与接口技术. [M]. 北京:清华大学出版社, 2004. 12
- [2] 郭天祥. 新概念 51 单片机 C 语言教程: 入门、提高、开发、拓展全攻略[M]. 电子工业出版社, 2009.
- [3] 康华光. 电子技术基础. 模拟部分[M]. 高等教育出版社, 1999.